

1. Template General

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define fast ios::sync_with_stdio(false); cin.tie(nullptr);
4     cout.tie(nullptr)
5 #define endl "\n"
6 #define pb push_back
7 #define all(x) (x).begin(), (x).end()
8 #define sz(x) (int)(x).size()
9
10 typedef long long ll;
11 typedef short int si;
12 typedef unsigned long long ull;
13 typedef long double ld;
14 typedef unsigned int ui;
15 typedef string str;
16 typedef pair<int, int> pii;
17 typedef pair<ll, ll> pll;
18
19 const int INFI = INT_MAX;
20 const long INFL = LONG_MAX;
21 const long long INFLl = LLONG_MAX;
22
23
24
25 int main(){
26     fast;
27
28     si n; cin >> n;
29     vector<int> v;
30     v.resize(n);
31     for (int i = 0; i < n; ++i) cin >> v[i];
32
33     cout << fixed << setprecision(10);
34
35     return 0;
36 }
```

2. Binary Exponentiation

```
1 ll binpow(ll a, ll b){
2     ll res = 1;
```

```

3     ll counter = 0;
4     while (b > 0){
5         bitset<16> binary(b);
6         if ( b& 1 ) res = res * a;
7         a + a * a;
8         counter++;
9         b >>=1;
10    }
11    return res;
12 }
13
14 ll binpowmod(ll a, ll b, ll m){
15     a \%= m;
16     ll res = 1;
17     while ( b > 0 ){
18         if ( b & 1 ) res = res * a \% m;
19         a = a * a \% m;
20         b >>= 1;
21     }
22     return res;
23 }

```

3. Swap dos números

```

1 void swap_two_iters(vector<int> vec, int n){
2     const auto itr = find(all(vec), n);
3     auto indx = distance(vec.begin(), itr);
4     iter_swap(vec.begin() + n, itr);
5     iter_swap(vec.begin() + n, vec.begin() + indx);
6 }

```

4. Bits

```

1     bitset<8> b1; // 00000000
2     bitset<8> b2(5); // 00000101
3     bitset<8> b3(string("1101")); // 00001101
4     bitset<8> b("10110010")
5     b[0] // 0 (least significant bit)
6     b[7] // 1 (most significant bit)
7     b[0] = 1; // Set bit 0 10110011
8     b.set(3); // set bit at index 3 10111011
9     b.reset(1); // reset bit at index 1
10    b.flip(2); // toggle bit at index 2 10111101

```

```
11  b.flip();
12
13  b.any() // True if any bit == 1
14  b.none() // true if all bits == 0
15  b.all() // true if all bits == 1
16  b.count() // number of bits == 1
17  b.size() // number of bits in bitset
18
19  b.to_ulong() // unsigned long
20  b.to_ullong() // unsigned long long
21  b.to_string() // string
```